

Ariel University
School of Computer Science

Serenity Shift

**A Therapeutic Simulation for Gradual
Exposure Training in Agoraphobia —
Delivered on Both PC and Virtual Reality**

Project Book

Submitted by:
Ori Hamou

Supervisor:
Prof. Erel Segal-Halevi

August 2026

Acknowledgements

I would like to thank my project supervisor, Prof. Erel Segal-Halevi, for his invaluable academic guidance and insightful perspective throughout this project. His honest critiques and creative suggestions were instrumental in shaping and improving the game.

A special thanks goes to our occupational therapy consultants — Nesya Zarbiv, Ifat Casovitz, and Coral Ben Baruch — occupational therapy students whose knowledge from their studies guided the design of the exposure hierarchy, the difficulty scaling, and the cognitive task system. Their feedback helped ground the game mechanics in therapeutic principles rather than in game-design intuition alone.

Finally, I would like to thank all the participants and testers who generously volunteered their time to try the game and complete the evaluation questionnaire. Their feedback and insights were crucial in refining the usability, pacing, and overall experience of *Serenity Shift*.

Contents

Acknowledgements	1
Abstract	4
1 Introduction	5
1.1 Background: Agoraphobia and Exposure Therapy	5
1.2 The Delivery Gap	5
1.3 Project Goals	6
1.4 Target Audience	6
2 Comparable Products and Approaches	7
3 System Overview	9
3.1 Two Modes of Play: PC and VR	9
3.2 The City Environment	9
3.3 Difficulty Tiers and Stressor Scaling	10
3.4 The Task Layer	10
3.5 Navigation and Accessibility Aids	10
3.6 Feedback, Timing, and Session Control	11
4 Architecture and Design	12
4.1 System Architecture	12
4.2 Technologies Used	12
4.3 The Two Interaction Systems	13
4.3.1 PC mode — mouse raycast	13
4.3.2 VR mode — XR Simple Interactable	13
4.3.3 Releasing a selected object safely	13
4.4 Scene and Level Structure	14
4.5 State Persistence	14
5 Development Process	15
5.1 Chronological Overview	15
5.2 Technical Challenges and Solutions	15
6 Key Features	17
6.1 Authentication and Progress Saving	17
6.2 NPC and Bird-Catching Mechanics	17
6.3 The Django Puzzle	17
6.4 Audio Systems	18
6.5 Navigation and Accessibility Aids	18
6.6 Traffic System	18

7	Testing and Evaluation	19
7.1	Evaluation Design	19
7.2	Quantitative Results	19
7.3	Qualitative Feedback	20
7.4	Public Presentation	21
7.5	Limitations	21
8	Results and Conclusions	22
A	Selected Code Samples	24
A.1	Bird-Catching Mechanic (VR) with Safe Release and Multi-Controller Fallback .	24
A.2	Weighted Random Selection Without Replacement (Bird Pool)	25
A.3	NPC Spawning with NavMesh Validation and Obstacle Rejection	25
A.4	Centralized Pause State Across Independent Systems	26
A.5	Traffic AI: Waypoint Navigation with Sensor-Based Obstacle Avoidance	26
B	Screenshots	28
C	Links	30

Abstract

Agoraphobia is an anxiety disorder in which individuals experience intense fear of open, crowded, or unfamiliar public spaces. Its severity spans a wide spectrum: for some it is mild discomfort in specific situations, such as avoiding busy supermarkets or crowded transport; for others it escalates into persistent avoidance of leaving home altogether, making everyday tasks feel overwhelming. The most established treatment is exposure therapy, in which patients are gradually and safely confronted with anxiety-inducing situations. However, real-world exposure is difficult to control, costly to arrange, and intimidating for patients to begin — and the avoidance that defines the condition often prevents patients from attempting it at all.

Serenity Shift addresses this gap with a therapeutic simulation that places the user in a realistic city environment as a controlled, gradual exposure tool. The user progresses through four difficulty tiers — tutorial, easy, medium, and hard — so exposure intensity can be tailored to each individual, scaling stressors such as pedestrian density, ambient city noise, and traffic. Interactive tasks — catching flagged pedestrians, a bird-catching objective, and an immersive street-performer puzzle (“Django”) with spatial 3D audio — keep the user actively and purposefully engaged within the feared space, redirecting attention away from anxiety-inducing stimuli while remaining within a therapeutic framework.

A defining characteristic of this project is that the same therapeutic core is delivered in **two interchangeable modes of play**: a **PC** version (keyboard and mouse, with a browser-playable WebGL build for maximum accessibility) and a **Virtual Reality** version (Meta Quest, with full presence and motion-controller interaction). Both modes share the same environment, the same difficulty progression, and the same task design; they differ only in how the user perceives and interacts with the world. This dual delivery lets the tool reach users who lack VR hardware while still offering the heightened immersion of VR where available.

The project was built in Unity with a full authentication and progress-saving system, difficulty-tiered scene management, and platform-specific interaction layers. It was presented at Expo TLV, Ariel University’s game-development showcase, and evaluated through a structured feedback questionnaire completed by 17 participants. Results were strongly positive: the overall concept was rated 4.7/5 on average, and 94% of respondents assigned the tool a concrete role within a therapeutic process. This report details the clinical motivation, the system design across both modes, the technical architecture, the development challenges, and the evaluation outcomes.

Project Availability: A browser-playable PC demo is available on itch.io, and a narrated video demonstration is available on YouTube. See Appendix C for all links.

Chapter 1

Introduction

1.1 Background: Agoraphobia and Exposure Therapy

Anxiety disorders are the most common class of mental-health conditions worldwide. Within this class, agoraphobia is defined by marked fear or anxiety about situations such as using public transportation, being in open or enclosed spaces, standing in a crowd, or being outside the home alone — situations the individual fears because escape might be difficult or help unavailable should panic-like symptoms occur [1]. The functional impact is severe and self-reinforcing: avoidance provides immediate relief from anxiety, which negatively reinforces the avoidance itself, until the person’s accessible world steadily contracts.

The clinical gold standard for treating agoraphobia is cognitive-behavioral therapy with **graded exposure**: the patient confronts a hierarchy of feared situations incrementally, allowing habituation and inhibitory learning to replace avoidance [2]. Simulation-based exposure therapy applies these same principles in a controlled, repeatable setting, and is typically delivered as a course of regular sessions in which the patient is gradually and repeatedly confronted with the feared situation.

1.2 The Delivery Gap

Despite its proven effectiveness, delivering exposure therapy in the real world is difficult for several structural reasons:

- **Controllability.** A real public space is inherently unpredictable — the number of people, the ambient noise, and unexpected events cannot be controlled or repeated from session to session. Exposure intensity therefore cannot be reliably calibrated to the patient’s current tolerance.
- **The avoidance paradox.** The core symptom of agoraphobia — avoidance — is precisely what prevents patients from attempting real-world exposure. Many refuse or drop out.
- **Cost and logistics.** Therapist-accompanied exposure outside the clinic is time-consuming and expensive, limiting how many sessions a patient realistically receives.
- **Attention diversion is hard to reproduce.** A useful exposure technique is redirecting the patient’s attention away from anxiety-inducing stimuli through a concrete task; this is difficult to arrange reliably in a real, uncontrolled environment.

A simulated environment resolves these directly: stress-inducing stimuli can be precisely and consistently adjusted, the scenario can be repeated identically, the patient can retreat or exit at any moment, and structured in-simulation tasks can be built in to direct attention. *Serenity Shift* is built on exactly these advantages.

1.3 Project Goals

The primary goal of *Serenity Shift* is to provide a controlled, repeatable exposure environment for individuals experiencing agoraphobia. Specific objectives:

- **Simulate a realistic public environment** — a busy city street — as the setting for gradual exposure exercises.
- **Enable precise control over stressors** — pedestrian density, ambient-sound intensity, and traffic — so exposure is scaled deliberately rather than left to real-world unpredictability.
- **Structure exposure into clear difficulty tiers** (tutorial, easy, medium, hard), so users progress gradually as tolerance builds.
- **Incorporate attention-diverting tasks**, giving users a practical technique for redirecting focus during exposure.
- **Deliver on accessible hardware in two modes** — a PC version requiring only a standard computer, and a VR version for headset users — maximizing reach without sacrificing immersion where hardware allows.
- **Support future clinical use** by laying groundwork for therapist-guided sessions and outcome tracking.

1.4 Target Audience

Serenity Shift is designed primarily for individuals with agoraphobia, from those with mild situational avoidance to those with more severe, isolating patterns, under a gradual, self-paced exposure model. Beyond direct patient use, it is relevant to mental-health professionals who may use it to support treatment plans, and it serves as a proof-of-concept for researchers interested in the intersection of interactive simulation and clinical exposure technique.

Chapter 2

Comparable Products and Approaches

A small number of existing tools apply related principles to exposure therapy for agoraphobia and anxiety disorders, though most differ significantly from *Serenity Shift*'s approach.

Agoraphobia Free (Thrive Therapeutic Software). A mobile CBT-based application using 3D characters and simulated situations. It guides the user through a *virtual character* experiencing agoraphobia, walking them through tasks based on cognitive restructuring and systematic desensitization rather than direct first-person exposure. Unlike *Serenity Shift*, the user observes and guides a character rather than personally navigating the feared environment.

SyMptOMS-ET. A geolocation-based smartphone application that guides patients through *in-vivo* exposure by directing them to real-world locations [3]. This relies on the patient physically visiting real places, which reintroduces exactly the unpredictability and logistical difficulty that simulation-based tools are designed to avoid.

App-guided exposure for panic/agoraphobia. Recent randomized controlled trials have evaluated mobile-app-guided exposure protocols for panic disorder with and without agoraphobia [4], confirming clinical interest in scalable, technology-delivered exposure — but these remain guidance-and-instruction tools rather than navigable simulated environments.

VR exposure platforms. Clinical VR platforms exist for anxiety treatment, but they tend to place the patient as a relatively passive observer of a feared scene, with little or no task layer and stimulus intensity driven by an operator rather than paced by the patient.

Distinguishing features of Serenity Shift. Against this landscape, *Serenity Shift*'s distinguishing features are: a fully simulated, first-person, navigable city environment; directly and deliberately adjustable stress variables (NPC density, ambient sound, traffic); structured difficulty progression; an active task layer (pedestrian catching, bird catching, the Django puzzle) that engages rather than merely exposes the user; and — uniquely — delivery of that same experience in **both a PC and a VR mode**, rather than committing to a single platform. This combination lets it serve users with and without VR hardware from one shared therapeutic design.

Table 2.1 summarizes how *Serenity Shift* compares with the approaches reviewed above across the dimensions most relevant to controlled exposure therapy.

Table 2.1: Comparison of exposure-therapy tools across key dimensions

	Agoraphobia Free	SyMptOMS -ET	VR platforms	Serenity Shift
First-person navigable simulation	No	No	Yes	Yes
Adjustable stress variables	Partial	No	Partial	Yes
Structured difficulty tiers	No	No	Partial	Yes
Active in-environment task layer	Partial	No	Partial	Yes
Patient-paced control	Yes	Partial	Partial	Yes
PC delivery	Yes*	Yes*	No	Yes
VR delivery	No	No	Yes	Yes

* Agoraphobia Free and SyMptOMS-ET are mobile (smartphone) applications; they run on consumer hardware but not as a desktop-PC experience. “Partial” indicates the capability exists in a limited or configuration-dependent form rather than as a core, directly-controlled feature.

The table makes the project’s position explicit: rather than out-scoring the alternatives on every dimension, *Serenity Shift* is the only tool reviewed that combines a navigable first-person simulation, deliberately adjustable stressors, structured difficulty, and an active task layer — and delivers that combination across *both* PC and VR, where each of the other approaches commits to a single form factor.

Chapter 3

System Overview

Serenity Shift builds Type-of-exposure therapy directly into an interactive experience: rather than keeping “therapy” and “play” separate, graded exposure is the core mechanic. This chapter describes the shared design that both modes present to the user.

3.1 Two Modes of Play: PC and VR

The single most important architectural fact about *Serenity Shift* is that it exists in **two interchangeable modes that share one therapeutic core**:

- **PC mode** — the user navigates the city with keyboard and mouse, selecting objects by pointing and clicking. A browser-playable WebGL build makes this mode reachable on almost any computer, with no installation.
- **VR mode** — the user is placed bodily inside the same city on a Meta Quest headset, looking and turning physically and selecting objects with a motion controller. This mode maximizes *presence* — the felt sense of really being in the feared space — which is the quality that makes exposure most effective.

Crucially, these are not two different products. They share the same city environment, the same four difficulty tiers, the same stressor scaling, the same tasks (pedestrian catching, bird catching, the Django puzzle), and the same navigation aids. What differs is only the *interface* between the user and that shared world: mouse-and-screen versus headset-and-controller. This design is deliberate and clinically motivated: VR delivers stronger immersion, but requires hardware many patients and clinics do not have; the PC mode ensures the tool remains accessible to everyone, while the VR mode offers a deeper experience where a headset is available. A patient might even begin on PC and progress to VR as a further step up the exposure ladder.

The technical consequences of supporting both modes — in particular the two different interaction systems — are detailed in Chapter 4.

3.2 The City Environment

The setting is a realistic American-style city street: multi-story buildings, sidewalks, storefronts, street furniture, vegetation, and open crossings. The environment was built from a modular city-construction asset and populated with pedestrian pathing routes. It is deliberately a single, consistent, learnable place — the user is not asked to master a new map at each stage, only to tolerate an increasingly demanding version of the same one.

3.3 Difficulty Tiers and Stressor Scaling

Exposure intensity is controlled through four tiers, which scale several independent stressor axes so the difficulty curve can rise gradually rather than all at once:

- **Tutorial** — an onboarding tier teaching core navigation and interaction (moving toward a marked objective) before exposure-based gameplay begins. Minimal stressors.
- **Easy** — calm, low pedestrian density, quiet ambience.
- **Medium** — denser crowds and louder city ambience; the bird- and pedestrian-catching objectives with positive-reinforcement feedback.
- **Hard** — maximum pedestrian density, the busiest soundscape, and — exclusive to this tier — a moving **traffic system** with vehicles and honking as an additional stressor, plus the Django puzzle as the tier objective.

The scaled axes are: **pedestrian (NPC) density and behavior**; **ambient-sound intensity** (from calm minimal ambience to dense crowd and city noise); **traffic presence** (introduced only at hard); and **mission complexity** (objectives scale alongside the environment). Progression is always at the user’s own pace — they select the tier appropriate to their current tolerance.

3.4 The Task Layer

Each exposure session is anchored by concrete, achievable tasks. Their therapeutic purpose is twofold: they redirect attention outward into the environment (counteracting the inward, self-monitoring focus that fuels panic), and each completed task is an unambiguous success experience *inside* the feared space — the ingredient that avoidance denies the patient.

- **Pedestrian (NPC) catching.** A subset of pedestrians is flagged as “catchable”; the user must pick them out from the ambient crowd and select them. This requires sustained attention and active engagement with the busy street rather than passive avoidance of it.
- **Bird catching.** At a different layer of the environment, the user locates and selects target birds. Each session, three birds are randomly selected from a pool of fifteen predetermined spawn locations, keeping the objective fresh across playthroughs. The mission birds use a distinct model so they are clearly identifiable as objectives, separate from the birds flying freely in the sky as ambient life.
- **The “Django” puzzle.** The hard-tier objective centers on Django, a street-performer character. Locating him emphasizes *audio* guidance: his music grows louder as the user approaches and quieter as they move away, encouraging navigation by ear. Once found, he presents a color sequence, and the user must press colored note-blocks — each producing a note of the solfège scale (do, re, mi, fa, sol, la, si) — in the correct order to reproduce it. After the puzzle is solved, his music continues at reduced volume for the rest of the session as pleasant ambience.

3.5 Navigation and Accessibility Aids

Because the city is large, a layered set of aids helps the user navigate confidently without becoming lost or overwhelmed — important for a therapeutic tool meant to build capability rather than frustration:

- **Compass** — an on-screen compass points toward the user’s nearest active objective at all times.
- **Objective particles** — a particle effect marks each objective’s location (yellow for pedestrians and birds), a visual cue complementing the compass.
- **Spatial 3D audio** — objective sounds grow louder with proximity, turning audio into a functional navigational tool, most notably in locating Django.

These three cues work together so users, regardless of gaming familiarity or anxiety level, can always find their way to the next objective.

3.6 Feedback, Timing, and Session Control

In the easy and medium tiers, finding an individual objective (a pedestrian or bird) triggers a chime and an encouraging **toast message** for immediate positive reinforcement. Every tier additionally shows a completion toast once its main goal is achieved — solving Django’s sequence, reaching the tutorial target, or finding all required objectives — reporting the total time taken (shown even if the on-screen timer was toggled off). A **pause menu** lets the user pause at any point, with persistent toggles for volume, ambient sound, timer, and compass display. The ability to pause or exit at any moment ensures the user always retains control over their exposure session — a core safety property for this population.

Chapter 4

Architecture and Design

4.1 System Architecture

Serenity Shift is a single-player Unity application structured around a shared city environment whose stressor intensity — pedestrian density, ambient sound, and traffic presence — is adjusted according to the selected difficulty tier. At a high level, the architecture consists of the following core modules, all shared between the PC and VR modes:

- **Player Controller** — handles movement, camera control, and interaction input. This is the primary module that differs between modes: a keyboard/mouse rig in PC mode and an XR rig with motion controllers in VR mode (Section 4.3).
- **Difficulty Manager** — governs which stressor parameters (NPC count, ambient-sound intensity, traffic presence) are active for the selected tier, and scales mission complexity accordingly.
- **NPC System** — manages pedestrian spawning, NavMesh-based pathfinding, and the subset of NPCs flagged as catchable for objectives.
- **Environmental Systems** — the ambient bird-flying system (sky life), the traffic system (active only at hard difficulty), and the city’s ambient soundscape.
- **Mission/Objective System** — tracks and manages objectives (pedestrian catching, bird catching, and the two-part Django encounter). Its purpose is to divert the user’s attention while they navigate an inherently stressful open public space, giving them a sense of accomplishment within that activity.
- **UI System** — the pause menu (persistent toggles for volume, ambient sound, timer, compass), the compass navigation aid, and the timer/performance display.
- **Authentication & Save System** — account authentication and cloud-based progress saving via Unity Gaming Services.

4.2 Technologies Used

- **Unity 6 (6000.2.8f1)** — the game engine, using the Universal Render Pipeline.
- **C#** — all gameplay logic, systems, and UI behavior.
- **Unity Gaming Services (Authentication & Cloud Save)** — account login and persistent progress saving.

- **XR Interaction Toolkit (3.2.2)**, **OpenXR (1.15.1)**, **Oculus XR plugin (4.5.2)**, over XR Plug-in Management (4.5.4), with the Input System (1.14.2) — the VR interaction stack for the Meta Quest build.
- **WebGL build pipeline** — exports a browser-playable version of the PC mode, optimized for size and full-screen compatibility across screens, including curved monitors.
- **Git / GitHub** — version control and repository management (Rehab-Game-Dev organization).

4.3 The Two Interaction Systems

Object selection — picking a catchable pedestrian, a target bird, or a Django note-block — is the most frequent user action, so its implementation defines the feel of each mode. Because PC and VR present fundamentally different input, the project implements **two interaction pathways that drive the same underlying task logic**:

4.3.1 PC mode — mouse raycast

In PC mode, selection uses a screen-to-world raycast: the mouse cursor position is projected into the scene, and a click resolves what was hit. This is the familiar point-and-click metaphor requiring no prior gaming experience.

4.3.2 VR mode — XR Simple Interactable

In VR mode, selection is built on Unity’s XR Interaction Toolkit. The motion controller carries an interactor that projects a ray into the scene and manages hover and selection states; each selectable object (bird, pedestrian, note-block) carries an **XR Simple Interactable** component. When the user points at a target and presses the trigger, the interactor raises a *select* event on that object, which the object handles by invoking its catch logic. The scene contains a single **XR Interaction Manager** that mediates all interactor–interactable communication.

A key design point is how each object connects its catch logic to the interactor’s event. Rather than wiring every object by hand in the Inspector — which would be error-prone and would not cover objects spawned at runtime — each catchable object subscribes itself to its own Simple Interactable’s select event in code, at initialization:

Listing 4.1: Self-subscribing a catchable object to the VR select event

```

1 private XRSimpleInteractable interactable;
2
3 private void Awake()
4 {
5     // Connect automatically to this object's own select event -
6     // works for pre-placed objects and runtime-spawned ones alike.
7     interactable = GetComponent<XRSimpleInteractable>();
8     interactable.selectEntered.AddListener(OnSelected);
9 }
10
11 private void OnSelected(SelectEnterEventArgs args) => CatchBird();

```

This keeps the therapeutic core — the environment, the tasks, the difficulty scaling — entirely shared, with the mode-specific difference isolated to how a selection is detected and dispatched.

4.3.3 Releasing a selected object safely

A subtlety specific to VR interaction is that when an object is caught and then destroyed while it is still the interactor’s currently selected object, the Toolkit can be left holding a reference

to a destroyed target, blocking subsequent selections. The catch logic therefore disables the object's interactable and collider *before* destroying it, so the interactor cleanly releases and is immediately free to select the next target. The full mechanic, and the debugging path that led to this solution, are described in Section 5.2; the code appears in Appendix A.

4.4 Scene and Level Structure

The city exists as difficulty-specific scene variants — `StreetScene - tutorial`, `StreetScene - easy`, `StreetScene - medium`, and `StreetScene - hard` — each authored with its own stressor configuration, plus menu scenes for level selection and, in the VR build, VR-specific menu scenes (`MainMenu_VR`, `EnvironmentMenu_VR`). This shared-environment, tiered-scene approach ensures visual and spatial consistency for the user as they progress: they adapt to an increasingly demanding version of one familiar place rather than learning a new map at each stage.

4.5 State Persistence

Player progression is saved to the cloud per user via Unity Cloud Save, so progress follows the account across sessions. In the PC standalone build, authentication and cloud save are fully available; the browser (WebGL) build of the PC mode does not include cloud-based persistence. A returning, authenticated user resumes at the tier they last selected — progress is written at the moment a tier is chosen, not only when a tier is completed, so a user resumes where they were rather than where they last finished.

Chapter 5

Development Process

5.1 Chronological Overview

Development followed a gradual, feature-by-feature progression, building the environment and gameplay systems before layering in polish and optimization.

Work began with constructing the **city environment** — streets, buildings, and overall layout — as the backdrop for every subsequent feature. Once the environment was in place, the **player character and navigation** were implemented, allowing free movement. With movement established, the **NPC system** was introduced, followed by the ambient **bird-flying system**, adding life to the streets and sky. Mission design then began with the **easy** mission, followed by the **medium** mission, after which the **tutorial** tier was built to onboard new users with the mechanics already in place.

Next, **cloud-based authentication** was integrated for account creation and progress saving, followed by the **traffic system**, and finally the **hard** mission, completing the four-tier structure. At this stage the project was presented at **Expo TLV**. Following the Expo, development shifted to refinement: a round of bug fixes, and quality-of-life features such as the full-featured pause menu and a level-loading bar. Development then extended the project to its **VR mode** on Meta Quest — adding the XR rig, motion-controller interaction, and VR-specific menus — and concluded with **WebGL build optimization** for the PC mode’s web deployment.

5.2 Technical Challenges and Solutions

NPC spawning and wall clipping. Early implementations occasionally spawned pedestrians inside walls or obstacles. This was resolved with a two-stage validation per candidate position: `NavMesh.SamplePosition` snaps a random point onto walkable ground, and `Physics.CheckSphere` then confirms the point is not physically obstructed — checked at two heights so both low walls and low overhangs are caught. A bounded retry count ensures the spawner gives up gracefully rather than looping forever in a crowded or mostly-unwalkable area (see Appendix A).

Pause-state persistence across scenes. Keeping pause-menu toggles (volume, ambient sound, timer) consistent across difficulty tiers — rather than resetting when the user switches tiers — required persisting UI state independently of any single scene’s lifecycle.

Difficulty-based mission scaling. How mission complexity should scale across the four tiers was guided by the occupational therapy students consulting on the project, drawing on the knowledge from their studies to help align the progression in stressor intensity and task difficulty with therapeutic exposure principles rather than a purely gameplay-driven difficulty curve.

Django model and guitar clipping. The Django character combined an animated body model with a separate guitar model; attaching them without z-fighting or mesh clipping required careful positioning so the guitar stays visually locked to the animation without artifacts.

Distance-based 3D audio for objective tracking. Beyond ambience, 3D audio needed a functional role: helping users locate objectives such as Django and the birds. This required tuning volume falloff and spatial parameters so intensity rises noticeably with proximity — neither too subtle to notice nor too jarring up close.

Navigation in a large city. Given the scale of the environment, users risked losing track of objectives. This was addressed by combining the compass (pointing to the nearest objective) with a particle effect at each objective, giving two complementary visual cues alongside the spatial audio.

VR interaction: reconciling custom selection with the XR Toolkit. The most involved VR-specific challenge concerned object selection. An initial approach performed a manual controller raycast that competed with the XR Interaction Toolkit’s own interactor system operating on the same objects; the two raced on the same physics scene, producing intermittent failures where targets would not respond or selections landed on the wrong object. The resolution was architectural: selection was consolidated onto the Toolkit’s interactor/interactable model, with each catchable object subscribing to its own **XR Simple Interactable** select event in code. A secondary defect — the first caught object blocking all subsequent selections — was traced to destroying an object while it was still the interactor’s selected target; disabling the interactable and collider before destruction lets the interactor release cleanly (Section 4.3, Appendix A).

WebGL build size. The initial WebGL build was significantly larger than needed for smooth web deployment. Through texture and audio compression, the build size was reduced from roughly 365 MB to approximately 150 MB, improving load times and accessibility across screen types, including curved monitors.

Chapter 6

Key Features

6.1 Authentication and Progress Saving

The application includes a full authentication system, allowing users to create an account and log in before a session, or to play as a **guest** with no account. Once authenticated, progress is saved via cloud storage, so sessions can be resumed and performance tracked over time. The guest path is handled defensively: because calling the Cloud Save API without an authenticated session would fail, progress is simply not persisted for guests rather than the game erroring. Cloud persistence is available in the standalone PC build; the browser (WebGL) build does not include it.

6.2 NPC and Bird-Catching Mechanics

The city is populated with non-player characters that behave as realistic pedestrians, using NavMesh-based pathfinding. Their behavior is consistent across tiers; only their *density* scales with difficulty, contributing to each session's stress intensity. Among them, a subset is flagged as **catchable**, forming the pedestrian-catching mission: the user must pick these out from the ambient crowd — a task requiring sustained attention and active engagement rather than passive avoidance.

The **bird-catching** mission follows the same principle at a different layer. Each session, three birds are randomly selected from a pool of fifteen predetermined spawn locations, keeping the objective fresh across playthroughs. Mission birds use a distinct model so they are clearly identifiable, separate from the ambient birds flying in the sky. Both bird and pedestrian objectives are highlighted with particle effects and tracked by the compass, so users can search purposefully without becoming lost.

Together these mechanics serve the core therapeutic goal: keeping the user actively and purposefully occupied within an open public space, redirecting attention away from anxiety-inducing stimuli through structured, achievable tasks.

6.3 The Django Puzzle

A distinct two-part encounter centers on **Django**, a street-performer character serving as the hard-tier objective. Locating him emphasizes audio guidance above the other cues: his music grows louder as the user approaches and quieter as they move away, encouraging active navigation by ear. Once found, he announces a riddle and speaks a sequence of colors; the objective then displays a set of written musical notes, each paired with its color. Colored note-blocks — each producing a note of the solfège scale (do, re, mi, fa, sol, la, si) when pressed — are located near Django, and the user must press them in the correct order to reproduce the sequence. After

the riddle is solved, Django’s music continues at reduced volume for the rest of the session as pleasant ambience.

6.4 Audio Systems

The city is built around a layered ambient audio system that contributes both to atmosphere and, more importantly, to the controlled scaling of stress across tiers. The ambient soundscape — general crowd and city noise — grows louder and busier at higher difficulties and calmer at lower ones, and can be toggled via the pause menu with the preference persisting across tiers. At the hard tier, the traffic system adds moving vehicles and honking as an additional stressor. Underlying much of this is **spatial 3D audio**, which turns sound into a functional navigational tool: cues grow louder as the user approaches a target such as Django, and fade as they move away.

6.5 Navigation and Accessibility Aids

As described in the overview, a **compass** points toward the nearest active objective, **particle effects** mark objective locations as a secondary visual cue, and **spatial audio** reinforces proximity through sound. Together these form a layered accessibility system ensuring users — regardless of gaming familiarity or anxiety level — can always find their way to the next objective within the city.

6.6 Traffic System

Exclusive to the hard tier, the traffic system adds vehicles that follow the city streets, combining deterministic route-following (advancing through a fixed list of waypoints via the NavMesh agent) with reactive obstacle sensing (a forward-facing raycast “bumper”). A car follows its route unless its sensor detects something ahead — another car or the user — in which case it halts and automatically resumes when the sensor clears. This produces believable traffic behavior, and the honking that accompanies it, as an additional environmental stressor at the highest intensity tier (see Appendix A).

Chapter 7

Testing and Evaluation

7.1 Evaluation Design

Player reception was evaluated through a structured feedback questionnaire. The intent was not to measure clinical symptom reduction — which would require a controlled longitudinal trial with diagnosed patients — but to answer the questions that must precede such a trial: is the environment believable, is the exposure experienced as gradual and controlled rather than overwhelming, are the tasks clear, and do users judge the tool relevant to a real therapeutic process?

The questionnaire was built in Hebrew using Google Forms and distributed to individuals who tried the game. It combined eight 5-point Likert items (1 = “to a very small extent”, 5 = “to a very great extent”), a multiple-choice item on where in a treatment process the tool would be integrated, and open-ended items on suggested improvements and additions. A total of **17 responses** were collected. Participants were a convenience sample recruited from the general public, without screening for clinical background or a personal history of anxiety; results therefore reflect *general users’ perceptions* of usability and therapeutic plausibility rather than expert clinical judgement, and the treatment-integration item in particular reflects lay intuition about therapeutic fit rather than professional assessment.

7.2 Quantitative Results

Table 7.1 and Figure 7.1 summarize the Likert results. All eight dimensions received mean ratings of 3.9 or higher, with the overall rating of the game and concept reaching **4.7/5** — every participant rated it 4 or 5.

Table 7.1: Likert-scale questionnaire results ($n = 17$; scale 1–5)

Dimension	Mean	Rated 4–5
Overall rating of the game and concept	4.7	100.0%
Comfort and naturalness of controls	4.4	82.4%
Environment size appropriate for the therapy	4.4	94.1%
Would recommend to a friend with agoraphobia	4.3	82.4%
Realism of the experience	4.2	76.5%
Difficulty and pacing suitability for patients	4.1	70.6%
Clarity of the missions	4.0	70.6%
Scenes’ fidelity to real agoraphobia triggers	3.9	70.6%

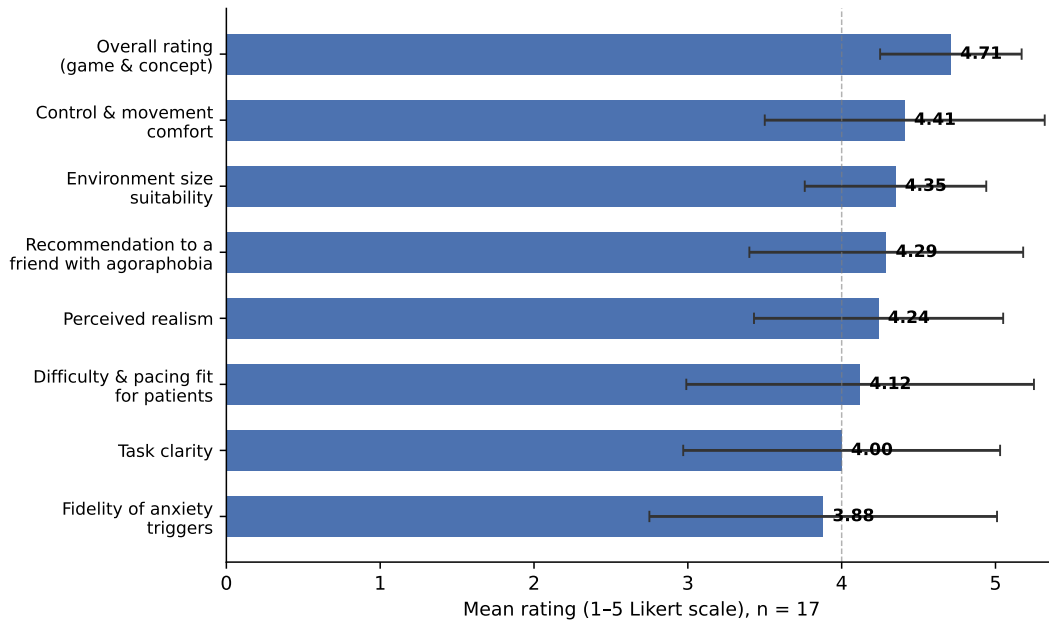


Figure 7.1: Mean participant ratings per evaluation dimension (error bars: ± 1 SD; dashed line at 4.0).

On where in a treatment process the tool would be used (Figure 7.2), most respondents — **12 of 17 (70.6%)** — said they would use it **alongside ongoing therapy**; **4 (23.5%)** would use it **before real-world exposure** as a preparatory bridge; and **1 (5.9%)** would not integrate it. In total, 94% assigned the tool a concrete role within a therapeutic process — and the two roles they gravitated toward (adjunct and bridge-to-in-vivo) are precisely those the exposure-therapy literature identifies for simulated exposure.

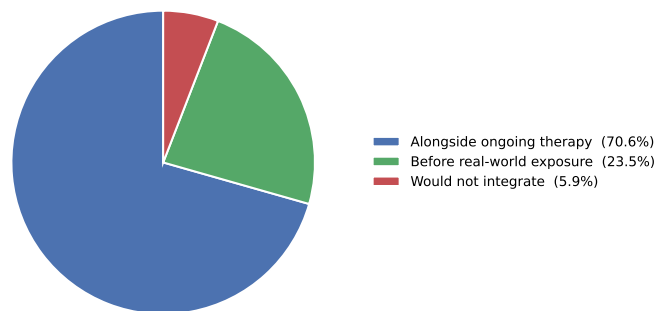


Figure 7.2: Distribution of answers to the treatment-integration question ($n = 17$).

7.3 Qualitative Feedback

Recurring themes in the free-text responses:

- **Controlled and gradual.** Multiple respondents described the exposure as controlled and gradual; one noted it felt slightly overwhelming at first but became appropriate as the session progressed — an accurate description of habituation, the mechanism exposure therapy relies on.

- **Graphics and UI polish.** The most common improvement suggestion concerned polishing the graphics and interface. This converges with the lowest-rated quantitative dimension (trigger fidelity, 3.9), suggesting higher visual fidelity is perceived as a path to greater therapeutic credibility.
- **Content expansion.** Several respondents suggested additional levels or scenarios tailored to other mental-health conditions beyond agoraphobia — an unprompted validation of the platform’s generality.
- **Gentler onboarding of aids.** One respondent recommended introducing the visual aids (compass, particle cues) more gradually, for a softer initial experience — fully aligned with the graded-exposure philosophy.

7.4 Public Presentation

Serenity Shift was presented at **Expo TLV**, Ariel University’s exhibition for the Computer Game Development course, where the games developed by students in the course were judged by faculty members from the university. The game was also seen by visitors attending the exhibition, and the overall response was positive.

7.5 Limitations

The evaluation has clear limitations, stated openly. The sample ($n = 17$) was small and recruited by convenience from the general public; participants were neither clinical professionals nor screened for agoraphobia. Results therefore reflect general users’ perceptions rather than clinical efficacy or expert judgement. The study was single-session with no baseline or control condition, and all measures were self-reported. Establishing actual symptom-level impact would require a controlled study with diagnosed patients, standardized anxiety instruments, and longitudinal follow-up — a natural next step beyond this project’s scope. Notably, the absence of clinically diagnosed participants was not for lack of effort: we reached out to numerous clinics and mental-health treatment providers in an attempt to recruit patients with agoraphobia, but were unable to secure their participation within the project’s timeframe. Evaluating the tool with diagnosed patients therefore remains an important direction for future work rather than a step that was overlooked.

Chapter 8

Results and Conclusions

Serenity Shift delivers on its core goal: a controlled, gradual, and engaging exposure simulation for individuals with agoraphobia, built around principles difficult to replicate in real-world sessions. It offers a fully navigable city with four difficulty tiers, each precisely scaling stressor intensity — pedestrian density, ambient sound, and traffic — while keeping mission complexity aligned with therapeutic exposure principles, with input from the occupational therapy students consulting on the project.

The project’s distinguishing achievement is delivering this same therapeutic core in **two modes of play** — PC and VR — from one shared design. The PC mode, with its browser-playable build, keeps the tool accessible to anyone with a computer; the VR mode offers the heightened presence that makes exposure most effective, for users with a headset. That both were built on one shared environment, difficulty system, and task design — differing only in the interaction layer — demonstrates a flexible, platform-independent architecture.

Beyond the core exposure mechanic, the project layers in features specifically supporting the user’s experience: attention-diverting missions (pedestrian catching, bird catching, the Django encounter), and accessibility aids (compass, particle cues, distance-based spatial audio) that prevent disorientation in a large open environment.

Feedback from 17 respondents was strongly positive, with an average overall rating of 4.7/5 and the majority indicating they would use the tool alongside ongoing therapy. This was reinforced by the reception at Expo TLV. Taken together, these results support the project’s central premise: that a simulation built with deliberate control over environmental stressors and guided by therapeutic design principles — and delivered flexibly across both PC and VR — can offer a viable, accessible, and engaging complement to traditional in-person exposure therapy for agoraphobia.

Bibliography

- [1] American Psychiatric Association. (2013). *Diagnostic and Statistical Manual of Mental Disorders* (5th ed.). Arlington, VA: American Psychiatric Publishing.
- [2] Craske, M. G., Treanor, M., Conway, C. C., Zbozinek, T., & Vervliet, B. (2014). Maximizing exposure therapy: An inhibitory learning approach. *Behaviour Research and Therapy*, 58, 10–23.
- [3] González-Pérez, A., Diaz-Sanahuja, L., Matey-Sanz, M., Osma, J., Granell, C., Bretón-López, J., & Casteleyn, S. (2024). Towards a self-applied, mobile-based geolocated exposure therapy software for anxiety disorders: SyMptOMS-ET app. *Digital Health*, 10. <https://doi.org/10.1177/20552076241283942>
- [4] Guth, M., Wiebe, A., Ekhlash-Schumann, M., Rodenjohn, A., Rohlfen, F., Buchholz, J., Selaskowski, B., Philipsen, A., & Braun, N. (2025). Mobile app-guided exposure therapy for panic disorder with and without agoraphobia: Randomized controlled trial. *Journal of Medical Internet Research*, 27, e76389. <https://doi.org/10.2196/76389>

Appendix A

Selected Code Samples

This appendix presents representative excerpts from the *Serenity Shift* codebase, chosen to illustrate the engineering concerns addressed during development: randomization, procedural placement, cross-system coordination, and the VR interaction mechanic central to this project.

A.1 Bird-Catching Mechanic (VR) with Safe Release and Multi-Controller Fallback

This component is attached to every catchable bird. It is the heart of the VR interaction and embodies several deliberate decisions.

```
1 public void CatchBird()
2 {
3     if (hasBeenCaught) return; // idempotency: trigger events can fire twice
4     // per frame
5     hasBeenCaught = true;
6
7     // Play the catch cue at the player's position (PlayClipAtPoint survives
8     // this
9     // object being destroyed a moment later).
10    if (catchSound != null)
11    {
12        GameObject player = GameObject.FindGameObjectWithTag("Player");
13        Vector3 pos = player != null ? player.transform.position : transform.
14        position;
15        AudioSource.PlayClipAtPoint(catchSound, pos);
16    }
17
18    // Different tiers use different message controllers; try each in turn.
19    if (GoalMessageController.Instance != null)
20        GoalMessageController.Instance.OnBirdCaught();
21    else if (BirdMessageController.Instance != null)
22        BirdMessageController.Instance.OnBirdCaught();
23
24    if (BirdGameManager.Instance != null)
25        BirdGameManager.Instance.BirdCaught();
26
27    // Release the XR interactor cleanly BEFORE destroying the object, so the
28    // next
29    // target can be selected immediately (see Chapter on challenges).
30    if (interactable != null) interactable.enabled = false;
31    Collider col = GetComponent<Collider>();
32    if (col != null) col.enabled = false;
33
34    Destroy(gameObject, 0.05f);
35 }
```

Three decisions are worth highlighting. The `hasBeenCaught` guard is an idempotency check: because trigger/select events can occasionally fire more than once for the same interaction within a single frame, the guard ensures a bird is only ever counted once. The message-controller fallback lets one reusable component work across tiers that use different progress-tracking systems — which controller is active is a scene-level configuration the script cannot know in advance, so it calls whichever singleton is actually present. Most importantly, the interactable and collider are disabled *before* `Destroy`: in VR, destroying an object while it is still the interactor’s selected target would leave the Toolkit holding a stale reference and block the next selection; disabling first forces a clean release. The small destruction delay gives the Toolkit a frame to process that release.

A.2 Weighted Random Selection Without Replacement (Bird Pool)

Each session activates a random subset of pre-placed bird objects as catchable targets.

```

1 void ActivateRandomBirds()
2 {
3     List<GameObject> availableBirds = new List<GameObject>(birdsInScene);
4     for (int i = 0; i < amountToSpawn; i++)
5     {
6         if (availableBirds.Count == 0) break; // pool smaller than requested:
            stop safely
7         int randomIndex = Random.Range(0, availableBirds.Count);
8         GameObject selectedBird = availableBirds[randomIndex];
9         selectedBird.SetActive(true);
10        availableBirds.RemoveAt(randomIndex); // remove so it can't be picked
            twice
11    }
12 }

```

This is sampling without replacement: rather than repeatedly picking a random index and risking a duplicate, each selected element is removed from a working copy before the next pick, guaranteeing distinct targets. The `break` handles the edge case where the requested amount exceeds the pool size, exiting early rather than throwing an out-of-range exception.

A.3 NPC Spawning with NavMesh Validation and Obstacle Rejection

Populating the street with pedestrians at valid, non-overlapping positions is harder than picking random points, because a random point has no guarantee of being walkable ground rather than the inside of a building.

```

1 public void SpawnNPCs()
2 {
3     int spawned = 0, attempts = 0, maxAttempts = amount * 10;
4     while (spawned < amount && attempts < maxAttempts)
5     {
6         attempts++;
7         Vector3 randomPoint = transform.position + Random.insideUnitSphere *
            range;
8
9         NavMeshHit hit;
10        if (!NavMesh.SamplePosition(randomPoint, out hit, 10.0f, NavMesh.
            AllAreas))
11            continue; // reject: nothing walkable near this point
12
13        Vector3 checkLow = hit.position + Vector3.up * 0.5f;
14        Vector3 checkMid = hit.position + Vector3.up * 1.5f;

```

```

15         if (Physics.CheckSphere(checkLow, clearanceRadius, buildingLayer) ||
16             Physics.CheckSphere(checkMid, clearanceRadius, buildingLayer))
17             continue; // reject: position is physically obstructed
18
19         GameObject npc = Instantiate(
20             npcPrefabs[Random.Range(0, npcPrefabs.Length)],
21             hit.position, Quaternion.Euler(0, Random.Range(0, 360), 0));
22         spawned++;
23     }
24     if (spawned < amount)
25         Debug.LogWarning($"NPCSpawner: Only spawned {spawned}/{amount} NPCs.");
26 }

```

Each candidate point passes two-stage validation: `NavMesh.SamplePosition` snaps it onto the nearest walkable surface (rejecting it if none is nearby), and `Physics.CheckSphere` confirms it is not obstructed — checked at two heights so both low walls and low overhangs are caught. The `attempts/maxAttempts` bound is defensive: without it, a mostly-unwalkable scene (or a request for more NPCs than the space can hold) would loop forever; instead it gives up gracefully and logs how many it placed.

A.4 Centralized Pause State Across Independent Systems

Pausing a real-time 3D game is a coordination problem: several independent systems not designed for pausing must all stop simultaneously.

```

1 public void Pause()
2 {
3     isPaused = true;
4     Time.timeScale = 0f; // freezes everything driven by Time.
5     pauseOverlay.SetActive(true);
6     Cursor.lockState = CursorLockMode.None;
7     Cursor.visible = true;
8
9     // NavMeshAgents ignore timeScale, so stop them explicitly.
10    foreach (NavMeshAgent agent in FindObjectsByType<NavMeshAgent>(
11        FindObjectsSortMode.None))
12        agent.isStopped = true;
13
14    // Pause only audio that is actually playing, so Resume() doesn't start
15    // silent sources.
16    pausedAudioSources.Clear();
17    foreach (AudioSource audio in FindObjectsByType<AudioSource>(
18        FindObjectsSortMode.None))
19        if (audio.isPlaying) { audio.Pause(); pausedAudioSources.Add(audio); }
20 }

```

`Time.timeScale = 0` freezes time-based systems automatically, but `NavMeshAgents` ignore `timeScale` and must be stopped explicitly, as must audio. The `pausedAudioSources` list is a deliberate detail: recording only sources that were actually playing means `Resume()` will not incorrectly start audio that was already silent.

A.5 Traffic AI: Waypoint Navigation with Sensor-Based Obstacle Avoidance

Vehicles at the hard tier combine deterministic route-following with reactive obstacle sensing in one update loop.

```

1 void CheckForObstacles()
2 {
3     RaycastHit hit;
4     Vector3 sensorStart = transform.position + transform.forward * 1.5f +
5         Vector3.up * 0.5f;
6     bool obstacleDetected =
7         Physics.Raycast(sensorStart, transform.forward, out hit,
8             detectionDistance, obstacleLayers)
9             && hit.collider.gameObject != gameObject; // ignore the car's own
10                collider
11
12     if (obstacleDetected)
13     {
14         agent.isStopped = true;
15         agent.velocity = Vector3.zero;
16         agent.angularSpeed = 0f;
17     }
18     else
19     {
20         agent.isStopped = false;
21         agent.angularSpeed = 120f;
22     }
23 }

```

A car follows its waypoint route unless the forward raycast “bumper” detects something ahead — another car or the player — in which case it halts by zeroing velocity and steering, resuming automatically when the sensor clears. This produces believable traffic without a full traffic-light system. The `hit.collider.gameObject != gameObject` check prevents a car from detecting its own collider and freezing permanently.

Appendix B

Screenshots

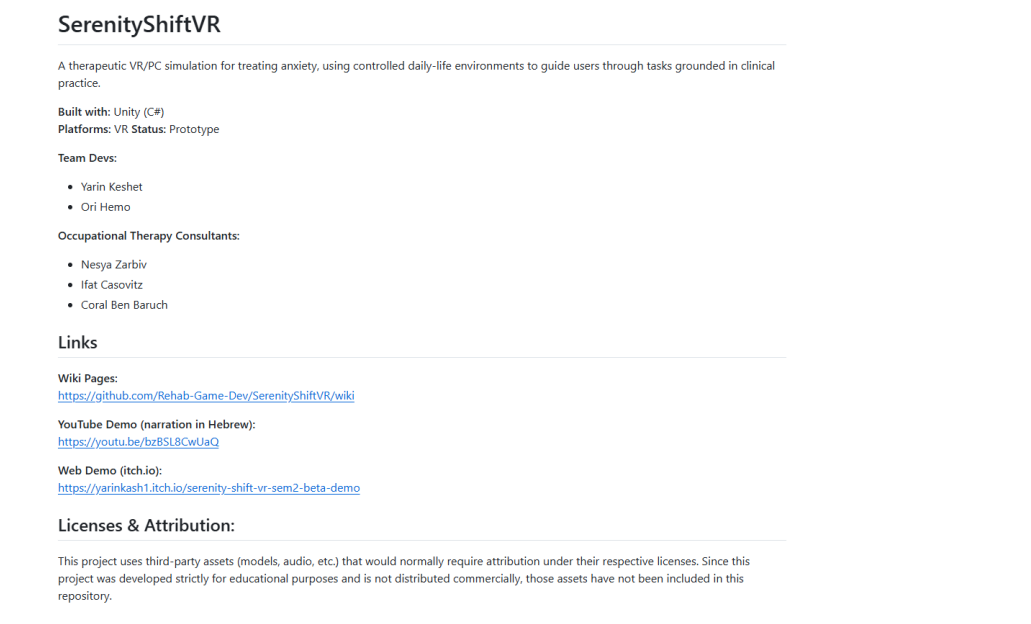


Figure B.1: The city street environment (PC mode), with the on-screen timer and objective banner. The user searches the open street for objectives.



Figure B.2: The bird-catching objective. The HUD (top-right) shows the current task and a progress counter; target birds are embedded naturally in the environment, requiring active scanning.



Figure B.3: The VR mode: the same city rendered for the Meta Quest headset, with the motion controller visible in-scene (bottom-right) and a target bird perched on the street lamp.

Appendix C

Links

- **GitHub Repository (VR):** <https://github.com/Rehab-Game-Dev/SerenityShiftVR>
- **Wiki:** <https://github.com/Rehab-Game-Dev/SerenityShiftVR/wiki>
- **YouTube Demo (narration in Hebrew):** <https://youtu.be/bzBSL8CwUaQ>
- **YouTube Trailer:** <https://youtube.com/shorts/lu3dgFSBwcg>
- **Web Demo (itch.io):** <https://yarinkash1.itch.io/serenity-shift-vr-sem2-beta-demo>